

# ESCUELA POLITÉCNICA NACIONAL

## Proyecto 1er Bimestre - Informe Técnico

**Integrantes:** Hernández Mark & Jiménez Yasid

**Materia:** Recuperación de la Información

**Docente:** Carrera Iván

### Librerías

```
In [1]: import kagglehub
import os
import pandas as pd
from src.preprocessor import TextPreprocessor
from src.models.tfidf import TfidfIR
from src.indexer import Indexer
from src.models.jaccard import JaccardIR
from src.models.bm25 import BM25IR
from src.models.embeddings import SemanticIR
from src.evaluator import Evaluator
from src.qrels import QrelsManager
```

```
c:\Users\mark_\Documents\Proyecto_ir26a\.venv\Lib\site-packages\tqdm\auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See http
s://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

### a. Construcción del índice

Esta parte es el corazón de nuestro proyecto. La idea es agarrar todos los documentos del corpus, limpiarlos y armar un **índice invertido** que nos permita buscar rápido después.

#### Puntos clave:

- Se descarga el dataset desde Kaggle usando `kagglehub`
- Se usan dos archivos: `ModApte_train.csv` (para entrenar) y `ModApte_test.csv` (para evaluar)
- Cada texto pasa por un preprocesamiento: limpieza → tokenización → stemming
- El resultado es un índice invertido: para cada término, sabemos en qué documentos aparece y cuántas veces

Necesario para recargar los cambios de los módulos de python

```
In [2]: %load_ext autoreload
%autoreload 2
```

Descargamos el corpus del dataset **thedevastator** de Kagglehub.

```
In [3]: # Download latest version
path = kagglehub.dataset_download("thedevastator/uncovering-financial-insights-w

print("Archivos en la carpeta descargada:")
for f in os.listdir(path):
    print(f)
```

Archivos en la carpeta descargada:

ModApte\_test.csv  
ModApte\_train.csv  
ModApte\_unused.csv  
ModHayes\_test.csv  
ModHayes\_train.csv  
ModLewis\_test.csv  
ModLewis\_train.csv  
ModLewis\_unused.csv

Para nuestro proyecto usaremos solo el archivo **ModApte\_train.csv** para entrenar nuestros modelos, ya que el resto son para testeo y variaciones del mismo dataset.

### Carga del dataset desde Kagglehub

```
In [4]: # Cargamos el set de ENTRENAMIENTO (ModApte_train.csv)
train_df = pd.read_csv(os.path.join(path, 'ModApte_train.csv'))

# Procesamos el corpus
train_df = train_df.dropna(subset=['text']).reset_index(drop=True)

corpus = train_df['text'].astype(str).tolist()
categorias = train_df['topics'].tolist()

# IDs de Los documentos
doc_ids = [f"Document_{i}" for i in range(len(corpus))]

print(f"Documentos cargados en Train: {len(corpus)}")
```

Documentos cargados en Train: 8816

### Carga del dataset desde un archivo local

```
In [5]: path = "data"

# Cargamos el set de ENTRENAMIENTO (ModApte_train.csv)
train_df = pd.read_csv(os.path.join(path, 'ModApte_train.csv'))

# Procesamos el corpus
corpus = train_df['text'].dropna().astype(str).tolist()

# IDs de Los documentos
doc_ids = [f"Document_{i}" for i in range(len(corpus))]

print(f"Documentos cargados en Train: {len(corpus)}")
```

Documentos cargados en Train: 8816

Creamos un dataframe que asocie a cada documento su contenido crudo y la categoría, misma que usaremos para saber que libros son relevantes acorde a las queries que vamos a proponer.

```
In [6]: df_corpus = pd.DataFrame({'doc_id': doc_ids, 'text_raw': corpus, 'categoria': ca}
df_corpus.head()
```

```
Out[6]:
```

|   | doc_id     | text_raw                                             | categoria                                             |
|---|------------|------------------------------------------------------|-------------------------------------------------------|
| 0 | Document_0 | Showers continued throughout the week<br>in\nthe ... | ['cocoa']                                             |
| 1 | Document_1 | The U.S. Agriculture<br>Department\nreported the ... | ['grain' 'wheat' 'corn' 'barley'<br>'oat' 'sorghum']  |
| 2 | Document_2 | Argentine grain board figures<br>show\ncrop regis... | ['veg-oil' 'linseed' 'lin-oil' 'soy-<br>oil' 'sun-... |
| 3 | Document_3 | Moody's Investors Service Inc said<br>it\nlowered... | []                                                    |
| 4 | Document_4 | Champion Products Inc said its\nboard<br>of direc... | ['earn']                                              |

Importamos los recursos necesarios para realizar el preprocesamiento del contenido de cada documento.

```
In [7]: # 2. Inicializar componentes
preprocessor = TextPreprocessor(language='english')
indexer = Indexer()
tfidf_model = TfidfIR(preprocessor=preprocessor, indexer=indexer)
```

Aplicamos la función transform para realizar todo el preprocesamiento: limpieza, tokenización y stemming.

```
In [8]: df_corpus['text_clean'] = df_corpus['text_raw'].apply(preprocessor.transform)
df_corpus.head()
```

Out[8]:

|   | doc_id     | text_raw                                          | categoria                                           | text_clean                                        |
|---|------------|---------------------------------------------------|-----------------------------------------------------|---------------------------------------------------|
| 0 | Document_0 | Showers continued throughout the week in\nthe ... | ['cocoa']                                           | shower continu throughout week bahia cocoa zon... |
| 1 | Document_1 | The U.S. Agriculture Department\nreported the ... | ['grain' 'wheat' 'corn' 'barley' 'oat' 'sorghum']   | us agricultur depart report farmerown reserv n... |
| 2 | Document_2 | Argentine grain board figures show\ncrop regis... | ['veg-oil' 'linseed' 'lin-oil' 'soy-oil' 'sun-...'] | argentin grain board figur show crop registr g... |
| 3 | Document_3 | Moody's Investors Service Inc said it\nlowered... | []                                                  | moodi investor servic inc said lower debt pref... |
| 4 | Document_4 | Champion Products Inc said its\nboard of direc... | ['earn']                                            | champion product inc said board director appro... |

Creamos el índice invertido a partir del vocabulario del corpus.

In [9]: `indexer.build_index(df_corpus, clean_text_column='text_clean', id_column='doc_id')`

Convertimos el índice invertido en un Dataframe para poder visualizar las frecuencias de cada término en los diferentes documentos.

In [10]:

```
# 1. Convertir el diccionario directamente en un DataFrame
# Al usar orient='index', las llaves (términos) se vuelven las filas (rows)
# y las llaves internas (Document_0, Document_5) se vuelven las columnas.
df_matrix = pd.DataFrame.from_dict(indexer.inverted_index, orient='index')

# 2. Reemplazar los valores NaN (donde la palabra no aparece en el documento) por 0
df_matrix = df_matrix.fillna(0)

# 3. Convertir el índice de términos en una columna normal llamada 'term'
df_matrix = df_matrix.reset_index().rename(columns={'index': 'term'})

# Visualizar el resultado
df_matrix.head(10)
```

Out[10]:

|   | term       | Document_0 | Document_3932 | Document_3949 | Document_3954 | Document_3955 |
|---|------------|------------|---------------|---------------|---------------|---------------|
| 0 | shower     | 1.0        | 1.0           | 1.0           | 5.0           | 0.0           |
| 1 | continuo   | 1.0        | 1.0           | 1.0           | 1.0           | 0.0           |
| 2 | throughout | 1.0        | 0.0           | 2.0           | 0.0           | 0.0           |
| 3 | week       | 4.0        | 1.0           | 1.0           | 3.0           | 0.0           |
| 4 | bahia      | 4.0        | 0.0           | 0.0           | 0.0           | 0.0           |
| 5 | cocoa      | 6.0        | 0.0           | 0.0           | 0.0           | 0.0           |
| 6 | zone       | 1.0        | 0.0           | 0.0           | 0.0           | 0.0           |
| 7 | allevi     | 1.0        | 0.0           | 0.0           | 0.0           | 0.0           |
| 8 | drought    | 1.0        | 0.0           | 0.0           | 0.0           | 0.0           |
| 9 | sinc       | 1.0        | 0.0           | 0.0           | 0.0           | 0.0           |

10 rows × 8817 columns



## b. Modelos de recuperación

Implementamos tres modelos distintos para buscar documentos dado un query. Cada uno tiene su forma de calcular qué tan relevante es un documento para la búsqueda.

### Los tres modelos:

- **Jaccard** — compara conjuntos de términos (binario: está o no está la palabra)
- **TF-IDF** — pondera términos raros más alto y mide ángulo entre vectores
- **BM25** — el más completo: controla repeticiones y longitud de documentos

Para probar brevemente todos los modelos se usa el mismo k=5 y la misma query.

```
In [11]: k=5
query = "Argentina economy"
```

### 1. Jaccard (vectores binarios)

**¿Qué hace?** Mira si los términos del query están en el documento. No le importa cuántas veces aparecen, solo si están o no. El score es: términos en común (intersección) / términos en total (unión). Simple pero no muy preciso con textos largos.

```
In [12]: # 1. Inicializamos y preparamos el modelo Jaccard
jaccard_model = JaccardIR(preprocessor=preprocessor, indexer=indexer)

# Ejecutamos su método index() para precalcular los términos únicos por document
jaccard_model.index()

# 2. Probamos la misma consulta para comparar resultados
```

```

print(f"\n--- Ejecutando consulta (Jaccard Binario): '{query}' ---")
resultados_jaccard = jaccard_model.search(query, k=k)

# 3. Mostrar Los resultados usando un DataFrame
filas_resultados = []

for rank, (doc_id, score) in enumerate(resultados_jaccard, 1):
    texto_original = df_corpus.loc[df_corpus['doc_id'] == doc_id, 'text_raw'].va

    # Creamos un diccionario por cada fila de resultado
    fila = {
        "Rank": rank,
        "Document ID": doc_id,
        "Similitud Jaccard": score,
        "Snippet": f"{texto_original[:150]}..." # Mantiene un fragmento corto c
    }
    filas_resultados.append(fila)

# Convertimos La Lista de filas en un DataFrame de Pandas
df_resultados = pd.DataFrame(filas_resultados)

display(
    df_resultados.style
        .format({"Similitud Jaccard": "{:.4f}"}) # Limita el score a 4 decimales
        .hide(axis="index") # Oculta el índice por defecto de
        .set_properties(**{'text-align': 'left'}, subset=['Snippet']) # Alinea el te
)

```

--- Ejecutando consulta (Jaccard Binario): 'Argentina economy' ---

| Rank | Document ID   | Similitud Jaccard | Snippet                                                                                                                                                   |
|------|---------------|-------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Document_1505 | 0.0370            | Argentina's cost of living index grew 6.5 pct in January, down from last month's 7.6 pct, the National Statistics Institute said. It said consumer...     |
| 2    | Document_2758 | 0.0286            | The White House welcomed the February retail sales figures showing a 4.1 pct rise, following a slow performance in January. Spokesman Marlin Fitzw...     |
| 3    | Document_4982 | 0.0270            | American Federal Savings and Loan Association of Colorado said its board cut the quarterly dividend to 7-1/2 cts per share from 15 cts. The divide...     |
| 4    | Document_2565 | 0.0263            | (Nationair) said it plans to begin service between Montreal's Mirabel airport and Brussels three times a week for an unrestricted one-way fare of 299 ... |
| 5    | Document_8410 | 0.0256            | The White House said the rise in interest rates was "unfortunate in a general sense" but reflected market forces. "There's always movement up and ...     |

## 2. TF-IDF con similitud coseno

**¿Qué hace?** Transforma cada documento en un vector de pesos. Los términos raros (que aparecen en pocos documentos) reciben más peso. Luego mide qué tan "ceranos" son el query y el documento usando el coseno del ángulo entre sus vectores (similitud coseno).

```
In [13]: # 1. Inicializamos y preparamos el modelo TF-IDF
tfidf_model = TfidfIR(preprocessor=preprocessor, indexer=indexer)

# Ejecutamos su método index() para calcular IDF y normas vectoriales
tfidf_model.index()

# 2. Probamos la misma consulta para comparar resultados
print(f"\n--- Ejecutando consulta (TF-IDF): '{query}' ---")
resultados_tfidf = tfidf_model.search(query, k=k)

# 3. Mostrar los resultados usando un DataFrame
filas_resultados_tfidf = []

for rank, (doc_id, score) in enumerate(resultados_tfidf, 1):
    texto_original = df_corpus.loc[df_corpus['doc_id'] == doc_id, 'text_raw'].va

    # Creamos un diccionario por cada fila de resultado
    fila = {
        "Rank": rank,
        "Document ID": doc_id,
        "Similitud Coseno": score, # Actualizado para reflejar la métrica de TF
        "Snippet": f"{texto_original[:150]}..."
    }
    filas_resultados_tfidf.append(fila)

# Convertimos la lista de filas en un DataFrame de Pandas
df_resultados_tfidf = pd.DataFrame(filas_resultados_tfidf)

display(
    df_resultados_tfidf.style
        .format({"Similitud Coseno": "{:.4f}"}) # Limita el score a 4 decimales
        .hide(axis="index") # Oculta el índice por defecto de
        .set_properties(**{'text-align': 'left'}, subset=['Snippet']) # Alinea el te
)
```

--- Ejecutando consulta (TF-IDF): 'Argentina economy' ---

| Rank | Document ID   | Similitud Coseno | Snippet                                                                                                                                                   |
|------|---------------|------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Document_3219 | 0.2690           | Central Bank President Jose Luis Machinea said negotiations with creditor banks on Argentina's 30 billion dlr private sector foreign debt were difficu... |
| 2    | Document_122  | 0.2636           | The U.S. Treasury said it was willing to participate with several other industrial countries in providing a 500 mln-dlr short-term bridge loan to Arge... |
| 3    | Document_969  | 0.2598           | Central Bank director Daniel Marx said Argentina's creditor banks were divided on the country's request for rescheduling its foreign debt. "There ...     |
| 4    | Document_6254 | 0.2527           | Central Bank President Jose Luis Machinea said Argentina could adopt the same posture as Brazil and suspend payments on its foreign debt if it failed ... |
| 5    | Document_4855 | 0.2503           | West German President Richard von Weizsaecker called on creditor banks to ease pressure on Argentina's foreign debt repayment terms. "We cannot re...     |

**¿Qué hace?** Es como TF-IDF pero mejorado. Tiene dos parámetros (  $k1=1.5$  ,  $b=0.75$  ) que evitan que un documento suba solo por repetir la misma palabra muchas veces, y también considera si el documento es muy largo o muy corto comparado al promedio del corpus.

```
In [14]: # 1. Inicializamos y preparamos el modelo BM25 (usa k1=1.5 y b=0.75 por defecto)
bm25_model = BM25IR(preprocessor=preprocessor, indexer=indexer)

# Ejecutamos su método index() para calcular el IDF probabilístico de BM25
bm25_model.index()

# 2. Probamos la misma consulta para comparar resultados
print(f"\n--- Ejecutando consulta (BM25): '{query}' ---")
resultados_bm25 = bm25_model.search(query, k=k)

# 3. Mostrar los resultados usando un DataFrame
filas_resultados_bm25 = []

for rank, (doc_id, score) in enumerate(resultados_bm25, 1):
    texto_original = df_corpus.loc[df_corpus['doc_id'] == doc_id, 'text_raw'].values[0]

    # Creamos un diccionario por cada fila de resultado
    fila = {
        "Rank": rank,
        "Document ID": doc_id,
        "Score BM25": score, # Actualizado para reflejar la métrica de BM25
        "Snippet": f"{texto_original[:150]}..."
    }
    filas_resultados_bm25.append(fila)

# Convertimos la lista de filas en un DataFrame de Pandas
df_resultados_bm25 = pd.DataFrame(filas_resultados_bm25)

display(
    df_resultados_bm25.style
        .format({"Score BM25": "{:.4f}"}) # Limita el score a 4 decimales
        .hide(axis="index") # Oculta el índice por defecto de Pandas
        .set_properties(**{'text-align': 'left'}, subset=['Snippet']) # Alinea el te
)
```

--- Ejecutando consulta (BM25): 'Argentina economy' ---



| Rank | Document ID   | Score BM25 | Snippet                                                                                                                                                   |
|------|---------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Document_3219 | 11.7280    | Central Bank President Jose Luis Machinea said negotiations with creditor banks on Argentina's 30 billion dlr private sector foreign debt were difficu... |
| 2    | Document_122  | 11.5916    | The U.S. Treasury said it was willing to participate with several other industrial countries in providing a 500 mln-dlr short-term bridge loan to Arge... |
| 3    | Document_969  | 11.2979    | Central Bank director Daniel Marx said Argentina's creditor banks were divided on the country's request for rescheduling its foreign debt. "There ...     |
| 4    | Document_4855 | 9.6392     | West German President Richard von Weizsaecker called on creditor banks to ease pressure on Argentina's foreign debt repayment terms. "We cannot re...     |
| 5    | Document_1677 | 9.4971     | A counter-proposal from creditor banks to Argentina's request for rescheduling its 50 billion dlr foreign debt was faulty, an Argentine observer at ne... |

## c. Interfaz básica

Se armó una interfaz sencilla para que cualquiera pueda hacer búsquedas sin tocar el código. Esta interfaz carga los modelos una sola vez al arrancar para que sea rápido y esta dentro del archivo **app.py** el cual debe ejecutarse con *streamlit run app.py*

### Puntos clave:

- Funciona desde la terminal. Tiene 5 secciones: Modelo Jaccard, Modelo TF-IDF, Modelo BM25, Embeddings, Evaluación.
- Los índices se cargan una sola vez (no recalcula en cada búsqueda).
- El usuario elige qué modelo usar y recibe el ranking estipulado con snippets del texto.

## d. Recuperación semántica con embeddings

Los modelos anteriores buscan palabras exactas. Este modelo entiende el *significado*. Convierte cada documento en un vector de alta dimensión (embedding) y busca por similitud semántica usando ChromaDB.

### Puntos clave:

- Usa un modelo preentrenado ( `sentence-transformers` ) para generar embeddings
- Los vectores se guardan en **ChromaDB** (base de datos vectorial)
- Busca por concepto, no por palabras exactas — entiende sinónimos y parafraseo
- Tarda más en indexar pero es el más potente para consultas en lenguaje natural
- Se usa el mismo k y query que para los modelos de recuperación

```

In [15]: # 1. Inicializamos el modelo semántico
# (Se conectará a ChromaDB y descargará el modelo preentrenado si es la primera
semantic_model = SemanticIR(df_corpus, text_column='text_clean', id_column='doc_

# 2. Generamos los embeddings del corpus y los guardamos en la base de datos vec
# Le pasamos batch_size=500 para que la memoria RAM no sufra
semantic_model.index(batch_size=500)

# 3. Ejecutamos la consulta de texto libre
print(f"\n--- Ejecutando consulta (Semántica con Embeddings): '{query}' ---")
# El método search ya se encarga de vectorizar la query y buscar en ChromaDB
resultados_semanticos = semantic_model.search(query, k=k)

# 4. Mostrar los resultados usando un DataFrame
filas_resultados_semanticos = []

for rank, (doc_id, score) in enumerate(resultados_semanticos, 1):
    texto_original = df_corpus.loc[df_corpus['doc_id'] == doc_id, 'text_raw'].va

    # Creamos un diccionario por cada fila de resultado
    fila = {
        "Rank": rank,
        "Document ID": doc_id,
        "Score Semántico": score, # Actualizado para reflejar la métrica de emb
        "Snippet": f"{texto_original[:150]}..."
    }
    filas_resultados_semanticos.append(fila)

# Convertimos la lista de filas en un DataFrame de Pandas
df_resultados_semanticos = pd.DataFrame(filas_resultados_semanticos)

display(
    df_resultados_semanticos.style
        .format({"Score Semántico": "{:.4f}"}) # Limita el score a 4 decimales
        .hide(axis="index") # Oculta el índice por defecto de Pa
        .set_properties(**{'text-align': 'left'}, subset=['Snippet']) # Alinea el te
)

```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF\_TOKEN to enable higher rate limits and faster downloads.

Loading weights: 100%|██████████| 103/103 [00:00<00:00, 1976.20it/s]

--- Ejecutando consulta (Semántica con Embeddings): 'Argentina economy' ---

| Rank | Document ID   | Score Semántico | Snippet                                                                                                                                                   |
|------|---------------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Document_5382 | 0.6358          | Argentina expects to reach agreement with its commercial bank creditors in the near future on a new financing package including 2.15 billion dlrs in n... |
| 2    | Document_122  | 0.6048          | The U.S. Treasury said it was willing to participate with several other industrial countries in providing a 500 mln-dlr short-term bridge loan to Arge... |
| 3    | Document_7740 | 0.5888          | U.S. Trade Representative Clayton Yeutter said the United States and Argentina have settled a case brought by the U.S. soybean crushing industry alle...  |
| 4    | Document_235  | 0.5796          | Argentina's chief debt negotiator said he was confident of a prompt accord with international creditor banks for rescheduling the country's foreign de... |
| 5    | Document_4855 | 0.5782          | West German President Richard von Weizsaecker called on creditor banks to ease pressure on Argentina's foreign debt repayment terms. "We cannot re...     |

## e. Evaluación de resultados

Comparamos los 4 modelos de forma objetiva. Se definen queries de prueba con sus documentos relevantes conocidos (qrels) y se miden métricas de recuperación.

### Puntos clave:

- Se usa **Precision** y **Recall** para medir qué tan buenos son los rankings según el k estipulado.
- Los 4 modelos se evalúan con las mismas queries. Esto nos permite comparar sus resultados.
- La tabla final muestra los valores del MAP de cada query asociada a cada modelo.

Antes de realizar la evaluación vamos a crear una clase **qrels.py** para poder realizar 3 cosas: poder crear un qrels de documentos relevantes para cada query asociado a cada modelo, poder mostrar esta estructura qrels de manera efectiva mediante un dataframe, y, poder sacar los documentos relevantes reales de los datos para enviarlos hacia la clase **evaluator.py**. Para ello creamos 10 consultas genéricas.

```
In [16]: # 1. Definimos las consultas
queries_prueba = {
    "Q01": "crude oil prices and OPEC production quotas",
    "Q02": "corporate quarterly earnings, revenue and profits",
    "Q03": "grain exports, wheat and corn supply supply",
    "Q04": "interest rates and federal reserve monetary policy",
    "Q05": "international trade disputes, tariffs and economic sanctions",
    "Q06": "gold and silver mining commodities market fluctuations",
    "Q07": "coffee and cocoa agricultural shipping shipments",
    "Q08": "economic inflation, consumer prices index and unemployment",
    "Q09": "stock market trading, mergers and corporate acquisitions",
    "Q10": "sugar production, cane refining and global trade quotas"
```

```

}

# 2. Agrupamos los modelos ya indexados
modelos = {
    "Jaccard": jaccard_model,
    "TF-IDF": tfidf_model,
    "BM25": bm25_model,
    "Semántico (Embeddings)": semantic_model
}

# 2. Instancias la clase qrels.py pasándole los datos base
gestor_qrels = QrelsManager(modelos_dict=modelos, queries_dict=queries_prueba)

# 3. Generamos los resultados (se guardan internamente en gestor_qrels.qrels_det
qrels_brutos = gestor_qrels.generar_qrels_por_modelos(k_pool=k)

# 4. Transformamos a un dataframe para una mejor visualización
df_comparativo = gestor_qrels.a_dataframe_comparativo()

```

Generando Qrels Detallados con k=5...

```

[Q01] 'crude oil prices and OPEC production quotas' procesada en todos los modelos.
[Q02] 'corporate quarterly earnings, revenue and profits' procesada en todos los modelos.
[Q03] 'grain exports, wheat and corn supply supply' procesada en todos los modelos.
[Q04] 'interest rates and federal reserve monetary policy' procesada en todos los modelos.
[Q05] 'international trade disputes, tariffs and economic sanctions' procesada en todos los modelos.
[Q06] 'gold and silver mining commodities market fluctuations' procesada en todos los modelos.
[Q07] 'coffee and cocoa agricultural shipping shipments' procesada en todos los modelos.
[Q08] 'economic inflation, consumer prices index and unemployment' procesada en todos los modelos.
[Q09] 'stock market trading, mergers and corporate acquisitions' procesada en todos los modelos.
[Q10] 'sugar production, cane refining and global trade quotas' procesada en todos los modelos.

```

Mostramos la visualización de los datos en **qrels** mediante el dataframe.

```
In [17]: df_comparativo.head(20)
```

Out[17]:

|   | Query ID | Consulta                                          | Rank | Jaccard       | TF-IDF        | BM25          | Sem (Embed |
|---|----------|---------------------------------------------------|------|---------------|---------------|---------------|------------|
| 0 | Q01      | crude oil prices and OPEC production quotas       | 1    | Document_239  | Document_239  | Document_239  | Documer    |
| 1 | Q01      | crude oil prices and OPEC production quotas       | 2    | Document_82   | Document_4075 | Document_1451 | Documer    |
| 2 | Q01      | crude oil prices and OPEC production quotas       | 3    | Document_1551 | Document_1451 | Document_2201 | Documer    |
| 3 | Q01      | crude oil prices and OPEC production quotas       | 4    | Document_4117 | Document_2596 | Document_4075 | Documer    |
| 4 | Q01      | crude oil prices and OPEC production quotas       | 5    | Document_2899 | Document_825  | Document_2710 | Documer    |
| 5 | Q02      | corporate quarterly earnings, revenue and profits | 1    | Document_8621 | Document_6985 | Document_6985 | Docum      |
| 6 | Q02      | corporate quarterly earnings, revenue and profits | 2    | Document_6512 | Document_789  | Document_789  | Documer    |
| 7 | Q02      | corporate quarterly earnings, revenue and profits | 3    | Document_1540 | Document_6460 | Document_6460 | Documer    |
| 8 | Q02      | corporate quarterly earnings, revenue and profits | 4    | Document_4247 | Document_7037 | Document_4631 | Documer    |

|    | Query ID | Consulta                                          | Rank | Jaccard       | TF-IDF        | BM25          | Sem (Embed |
|----|----------|---------------------------------------------------|------|---------------|---------------|---------------|------------|
| 9  | Q02      | corporate quarterly earnings, revenue and profits | 5    | Document_6985 | Document_779  | Document_779  | Documer    |
| 10 | Q03      | grain exports, wheat and corn supply supply       | 1    | Document_5808 | Document_5808 | Document_5808 | Docume     |
| 11 | Q03      | grain exports, wheat and corn supply supply       | 2    | Document_832  | Document_2091 | Document_2091 | Documer    |
| 12 | Q03      | grain exports, wheat and corn supply supply       | 3    | Document_7010 | Document_8016 | Document_1061 | Documer    |
| 13 | Q03      | grain exports, wheat and corn supply supply       | 4    | Document_6556 | Document_2955 | Document_834  | Documer    |
| 14 | Q03      | grain exports, wheat and corn supply supply       | 5    | Document_4663 | Document_6556 | Document_2955 | Docume     |
| 15 | Q04      | interest rates and federal reserve monetary po... | 1    | Document_8699 | Document_8666 | Document_8670 | Documer    |
| 16 | Q04      | interest rates and federal reserve monetary po... | 2    | Document_8666 | Document_8670 | Document_8666 | Documer    |
| 17 | Q04      | interest rates and federal reserve                | 3    | Document_4716 | Document_5708 | Document_1350 | Documer    |

|    | Query ID | Consulta                                          | Rank | Jaccard       | TF-IDF        | BM25          | Sem (Embed |
|----|----------|---------------------------------------------------|------|---------------|---------------|---------------|------------|
|    |          | monetary po...                                    |      |               |               |               |            |
| 18 | Q04      | interest rates and federal reserve monetary po... | 4    | Document_7280 | Document_1350 | Document_2933 | Documer    |
| 19 | Q04      | interest rates and federal reserve monetary po... | 5    | Document_8183 | Document_2933 | Document_5708 | Documer    |

Con esta evidencia de que libros recupera cada modelo en su top 3 para cada query, procedemos a crear el qrels verdadero basándonos en la columna *Category* de nuestro dataframe *df\_corpus*. Posterior a ello, realizamos la evaluación con este **qrels** adecuado.

```
In [18]: # 4.5 Definir el mapeo de consultas a etiquetas del dataset
mapeo_consultas_categorias = {
    "Q01": "crude",
    "Q02": "earn",
    "Q03": "grain",
    "Q04": "interest",
    "Q05": "trade",
    "Q06": "gold",
    "Q07": "coffee",
    "Q08": "cpi",          # Ajusta esto a 'inflation' si tu dataset lo usa así
    "Q09": "acq",
    "Q10": "sugar"
}

# 5. Generar el verdadero Terreno de Verdad desde el corpus
qrels_para_evaluar = gestor_qrels.generar_ground_truth_desde_corpus(
    df_corpus=df_corpus,
    mapeo_categorias=mapeo_consultas_categorias
)

# 6. Inicializar el evaluador con el terreno de verdad (ground truth)
evaluador = Evaluator(qrels_para_evaluar)

# 7. Obtener la métrica global del sistema (MAP)
df_map_global, _ = evaluador.evaluar_modelos(modelos, queries_prueba, k=k)
df_map_global["MAP"] = df_map_global["MAP"].round(4)

# 8. Extraer métricas agregadas (Promedio de Precision y Recall) por modelo
filas_metricas_agregadas = []

for nombre_modelo, modelo in modelos.items():
    precisiones = []
    recalls = []
```

```

for q_id, q_text in queries_prueba.items():
    # Ejecutamos la búsqueda simulando la evaluación al k definido
    resultados = modelo.search(q_text, k=k)

    # Calculamos Precision y Recall de esta consulta específica
    p, r = evaluador.precision_recall(resultados, q_id)

    # Guardamos los valores para promediarlos luego
    precisiones.append(p)
    recalls.append(r)

    # Calculamos el promedio para el modelo actual
    avg_precision = sum(precisiones) / len(precisiones) if precisiones else 0
    avg_recall = sum(recalls) / len(recalls) if recalls else 0

    filas_metricas_agregadas.append({
        "Modelo": nombre_modelo,
        f"Precision@{k}": round(avg_precision, 4),
        f"Recall@{k}": round(avg_recall, 4)
    })

df_metricas_agregadas = pd.DataFrame(filas_metricas_agregadas)

# 9. Unificar todas las métricas globales en un solo DataFrame
df_resumen_total = pd.merge(df_map_global, df_metricas_agregadas, on="Modelo")

# =====
# VISUALIZACIÓN DEL DATAFRAME FINAL
# =====

print("\n--- Rendimiento Global por Modelo ---")
display(df_resumen_total)

```

--- Extrayendo Terreno de Verdad (Ground Truth) del Corpus ---

[Q01] Categoría 'crude': 353 documentos reales encontrados.  
 [Q02] Categoría 'earn': 2725 documentos reales encontrados.  
 [Q03] Categoría 'grain': 399 documentos reales encontrados.  
 [Q04] Categoría 'interest': 291 documentos reales encontrados.  
 [Q05] Categoría 'trade': 339 documentos reales encontrados.  
 [Q06] Categoría 'gold': 94 documentos reales encontrados.  
 [Q07] Categoría 'coffee': 110 documentos reales encontrados.  
 [Q08] Categoría 'cpi': 60 documentos reales encontrados.  
 [Q09] Categoría 'acq': 1490 documentos reales encontrados.  
 [Q10] Categoría 'sugar': 119 documentos reales encontrados.

--- Rendimiento Global por Modelo ---

|   | Modelo                 | MAP    | Precision@5 | Recall@5 |
|---|------------------------|--------|-------------|----------|
| 0 | BM25                   | 0.0262 | 0.88        | 0.0262   |
| 1 | TF-IDF                 | 0.0240 | 0.84        | 0.0243   |
| 2 | Jaccard                | 0.0195 | 0.74        | 0.0209   |
| 3 | Semántico (Embeddings) | 0.0188 | 0.80        | 0.0217   |

## f. Comparación de modelos y Análisis de Resultados



Tras evaluar los cuatro sistemas de recuperación implementados (Jaccard, TF-IDF, BM25 y Embeddings Semánticos) contra el corpus Reuters-21578, se observan diferencias marcadas que confirman la teoría de la recuperación de la información, evidenciadas en nuestras métricas de **MAP** y **Precision@5**.

## 1. Análisis de Rendimiento Cuantitativo

Basándonos en la tabla de evaluación global obtenida, podemos destacar lo siguiente:

- **BM25 es el líder indiscutible:** Logró el mejor rendimiento con un **MAP de 0.0262** y una **Precisión@5 sobresaliente de 0.88**. Esto confirma que su mecanismo de saturación de frecuencia y penalización por longitud de documento es el más robusto para este tipo de corpus periodístico/financiero.
- **TF-IDF como sólido segundo lugar:** Con un **MAP de 0.0240** y **Precisión@5 de 0.84**, demuestra que la ponderación de términos raros (IDF) es altamente efectiva para consultas específicas basadas en palabras clave.
- **Jaccard muestra sus limitaciones:** Aunque mantiene una Precisión@5 decente (0.74), su **MAP (0.0195)** refleja cómo el modelo binario sufre al no considerar la frecuencia de los términos, viéndose fuertemente penalizado por documentos largos.

## 2. ¿En qué tipos de consultas funciona mejor cada modelo?

- **Modelo Binario (Jaccard):** Funciona mejor en consultas muy cortas y específicas (1 o 2 palabras) donde solo importa la presencia del término. Tiende a fallar en documentos largos, ya que la penalización por la longitud del documento (la unión de conjuntos) diluye rápidamente el score.
- **Modelo Vectorial (TF-IDF):** Es ideal para consultas que contienen términos muy raros o específicos. Al usar una escala logarítmica para el IDF, le da un peso altísimo a esas palabras poco comunes, asegurando que los documentos que las contienen suban en el ranking.
- **BM25:** Es el modelo basado en términos más robusto. Destaca en consultas de longitud media y sobre corpus con documentos de tamaños muy variables.

## 3. El desempeño de la Recuperación Semántica (Embeddings)

Curiosamente, el modelo **Semántico obtuvo el MAP más bajo (0.0188)** y una Precisión@5 (0.80) inferior a los modelos estadísticos avanzados (BM25 y TF-IDF). Lejos de ser un error, este resultado es un reflejo perfecto de la naturaleza de nuestro experimento:

**La recuperación semántica EMPEORA en este escenario porque:**

- **Naturaleza del Dataset y las Queries:** El corpus de Reuters es financiero y altamente técnico. Nuestras consultas (ej. *"crude oil prices and OPEC production quotas"*) buscan entidades nombradas exactas y jerga económica precisa.
- **Falta de Exact Match:** Cuando se buscan acrónimos o términos exactos, los modelos semánticos suelen perder precisión y traen documentos que están "cerca" temáticamente, pero que no responden a la palabra clave exacta. En estos casos de

coincidencia exacta (*exact match*), **BM25 y TF-IDF superan ampliamente** al modelo semántico, ya que van directo a la lista de posteo de ese token único.

**Sin embargo, la recuperación semántica MEJORARÍA si:**

- **Existiera desajuste de vocabulario (Vocabulary Mismatch):** Si la consulta fuera en lenguaje más natural o ambiguo (ej. "*crisis económica en un país de sudamérica*" para encontrar documentos sobre "*Argentina sufre inflación*"), BM25 y TF-IDF fracasarían rotundamente por falta de tokens coincidentes, mientras que los embeddings lograrían capturar la relación conceptual.